

PRINCIPLES OF PROGRAMMING LANGUAGES / LESSON GUIDE

07 Syntax error

Modules 1-2: tokens, grammars, parsing, and diagnostics

1. Introduction & core concept

Syntax specifies the forms of expressions, statements, and declarations admitted by a language's grammar. A syntax error occurs when source cannot be accepted under the applicable syntactic rules. A program may use individually recognizable tokens yet combine them into an invalid structure. Syntax is distinct from intended meaning, type correctness, and successful execution.

Language design uses grammar to make structure sufficiently precise for both humans and implementations. Delimiters, indentation, precedence, and statement terminators help resolve boundaries and grouping. Different languages choose different conventions; forcing a universal visual style can obscure rather than improve their syntax.

2. Under the hood: the mechanics

Lexical analysis recognizes tokens; parsing attempts to build a structured representation such as an AST. A diagnostic marks where acceptance became impossible, which may be after the actual mistake. Missing delimiters or terminators can cause later text to be interpreted in the wrong context. Compiler recovery can then produce several follow-on messages from one defect.

Parsing can happen before a file runs or later when a running program compiles another source string. Python compile and JavaScript Function can therefore report syntax errors as exceptions inside an already running host program [1-2]. In C++, a compilation diagnostic can instead concern declarations, types, or other semantic constraints; 'compiler error' is a broader category than 'syntax error'.

Learning outcomes

- Locate syntax in the translation pipeline.
- Distinguish parse-time rejection from type and runtime failures.
- Read the earliest useful diagnostic and recheck after one repair.

Examples progress from Python 3 to modern JavaScript to C++17. Code follows each language's native syntax. Bracketed citations refer to the final page.

3. CODE EXAMPLES / PYTHON

Simple: inspect a malformed function header

```
source = "def broken(:\n    return 1\n"

try:
    compile(source, "student.py", "exec")
except SyntaxError as error:
    print(type(error).__name__)
    print(error.lineno)

fixed = "def corrected():\n    return 1\n"
compile(fixed, "student.py", "exec")
print("Corrected source parses")
```

In-depth reasoning

Expected result: SyntaxError

1

Corrected source parses

The host program itself is valid Python. `source` contains another program as text. Calling `compile` asks Python to parse and compile that text without executing it. The malformed parameter list fails on line one, and the host catches the resulting `SyntaxError`. The exact message and caret placement can vary by Python version, so the example prints stable diagnostic fields.

In `fixed`, the empty parameter list closes before the colon, and the body is indented consistently. `compile` accepts this source and returns a code object, which the example discards. Acceptance does not call `corrected` or prove that every possible execution would succeed. Parsing and execution remain separate actions.

A syntax error directly in the host file would prevent this `try` statement from executing at all. The apparent paradox disappears once the two compilation units are distinguished: one is already running, and it is asking the implementation to process another. Never use this pattern to execute untrusted student code in an unrestricted host process.

Compare the language models

JavaScript can likewise dynamically parse fixed source strings and catch `SyntaxError`. A conventional C++ build rejects malformed translation units before producing the intended executable. Runtime error handling cannot rescue a source file that never successfully became the running program.

Checkpoint: Does successful `compile` establish that a function returns the desired answer? No; it establishes acceptance at compilation, not behavioral correctness.

3. CODE EXAMPLES / JAVASCRIPT

Intermediate: parsing during execution

```
try {  
    new Function("return (1 + 2);");  
} catch (error) {  
    console.log(error.name);  
}  
  
const calculate = new Function("return (1 + 2);");  
console.log(calculate());
```

In-depth reasoning

Expected result: SyntaxError
3

The first constructor call requests a function body from a fixed string. Its opening parenthesis has no closing partner before the semicolon, so construction fails with `SyntaxError`. The catch belongs to the valid host script and can observe this dynamic parsing failure. There is no partially valid `calculate` function produced by that first call.

The second string contains a balanced expression and a return statement. Function construction succeeds; invoking the resulting function evaluates the addition and returns 3. The constructor creates a function in the global environment rather than capturing the caller's local lexical environment, a distinction from an ordinary nested function [2].

Dynamic source construction is used here only to make the parser boundary visible. It is not recommended as an input-validation technique or as a way to process untrusted expressions. A quoted piece of text is data until an operation explicitly asks the implementation to interpret it as source.

Compare the language models

Python compile can produce a code object without running it; execution requires a separate step. JavaScript `Function` produces a callable that is then invoked. C++ commonly translates files ahead of execution. Error timing depends on which unit is being processed, not just the language's interpreted/compiled label.

Checkpoint: Would `try` catch a missing parenthesis written directly into this host script? No; the host script would fail to parse before its handler could run.

3. CODE EXAMPLES / C++

Advanced: an intentional declaration terminator error

```
#include <iostream>

struct Point {
    int x;
} // Intentional error: the declaration needs a semicolon.

int main() {
    Point point{7};
    std::cout << point.x << '\n';
}
```

In-depth reasoning

Expected result: Expected: compilation fails. After adding ';' after the struct, output is 7.

This block is intentionally not a valid complete C++ program. The closing brace ends the class body, but the declaration still requires its semicolon. The compiler may point at that brace or nearby following text because that is where its interpretation can no longer continue. Exact diagnostic wording depends on the compiler.

Repair the declaration by placing a semicolon immediately after its closing brace. The aggregate initialization `Point point{7}` then initializes `x`, and the stream insertion displays 7. No special error-catching code in `main` could repair the original translation failure: `main` never becomes executable from this malformed unit.

Now consider a different change: referring to `point.missing` after fixing the terminator. That is structurally a member-access expression, but the requested member does not exist. A compiler rejects it through semantic analysis. Both cases are compile failures, but identifying the phase helps explain the rule violated and choose the correct repair.

Compare the language models

Python uses indentation and colons for suites; JavaScript and C++ use braces for many blocks. C++ additionally distinguishes a class definition's closing brace from the declaration terminator. The correct repair follows native grammar, not a visual resemblance to another language.

Checkpoint: Why fix the first meaningful diagnostic before all later ones? Parser recovery can turn one missing token into many misleading follow-on messages.

4. CONSOLIDATION / REFERENCES

Reason, compare, and apply

The central lesson

A reliable diagnostic workflow is to identify the unit being parsed, inspect the earliest useful message, check surrounding delimiters and indentation, repair one structural cause, and rerun. Paradigm Diagnostics uses Python's parser for Python evidence; JavaScript/C++ surface metrics cannot certify complete grammar acceptance. Run invokes their actual toolchains.

Review questions with answer guidance

1. Can a syntax error be observed while another program runs?

Yes, when the running program requests parsing or compilation of another source unit.

2. Does a valid AST prove type correctness or termination?

No. Parsing supplies structure; further checks and execution determine other properties.

A short learning activity

Predict each example's outcome before running it. Change one input or one operation, state the rule you expect to apply, and compare the observation with the prediction. Explain the earliest point where behavior changes. A matching output supports that particular execution; it does not prove general program equivalence.

References and further reading

[1] Python: compile. [Read source](#)

[2] ECMAScript: dynamic function creation. [Read source](#)

[3] C++ working draft: declarations. [Read source](#)

References are official language/runtime documentation or the public C++ working draft. Links checked 30 September 2026. Examples and reasoning are original teaching material; implementation details and scheduling outcomes are identified where relevant.